

AG2 - Actividad Guiada 2

Nombre: Guido Quispe Quispe

Link:

Github: https://github.com/Guidouib/Algoritmos_de_Optimizacion/tree/main/AG2%20-%20Actividad%20Guiada%202

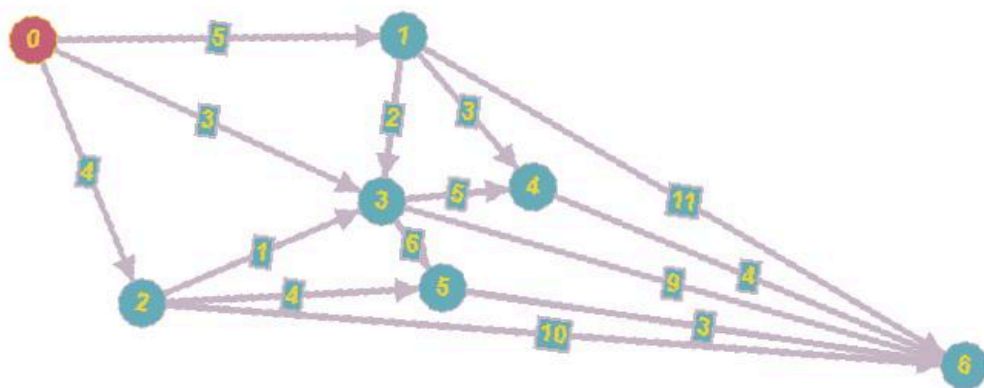
In [58]: `import math`

Programación Dinámica. Viaje por el río

- **Definición:** Es posible dividir el problema en subproblemas más pequeños, guardando las soluciones para ser utilizadas más adelante.
- **Características** que permiten identificar problemas aplicables:
 - Es posible almacenar soluciones de los subproblemas para ser utilizados más adelante
 - Debe verificar el principio de optimalidad de Bellman: "en una secuencia optima de decisiones, toda sub-secuencia también es óptima" (*)
 - La necesidad de guardar la información acerca de las soluciones parciales unido a la recursividad provoca la necesidad de preocuparnos por la complejidad espacial (cuantos recursos de espacio usaremos)

Problema

En un río hay n embarcaderos y debemos desplazarnos río abajo desde un embarcadero a otro. Cada embarcadero tiene precios diferentes para ir de un embarcadero a otro situado más abajo. Para ir del embarcadero i al j , puede ocurrir que sea más barato hacer un trasbordo por un embarcadero intermedio k . El problema consiste en determinar la combinación más barata.



*Consideramos una tabla TARIFAS(i,j) para almacenar todos los precios que nos ofrecen los embarcaderos.

*Si no es posible ir desde i a j daremos un valor alto para garantizar que ese trayecto no se va a elegir en la ruta óptima(modelado habitual para restricciones)

```
In [59]: #Viaje por el rio - Programación dinámica
#####

TARIFAS = [
[0,5,4,3,float("inf"),999,999], #desde nodo 0
[999,0,999,2,3,999,11], #desde nodo 1
[999,999, 0,1,999,4,10], #desde nodo 2
[999,999,999, 0,5,6,9],
[999,999, 999,999,0,999,4],
[999,999, 999,999,999,0,3],
[999,999,999,999,999,999,0]
]

#999 se puede sustituir por float("inf") del modulo math
TARIFAS
```

```
Out[59]: [[0, 5, 4, 3, inf, 999, 999],
[999, 0, 999, 2, 3, 999, 11],
[999, 999, 0, 1, 999, 4, 10],
[999, 999, 999, 0, 5, 6, 9],
[999, 999, 999, 999, 0, 999, 4],
[999, 999, 999, 999, 999, 0, 3],
[999, 999, 999, 999, 999, 999, 0]]
```

```
In [60]: #Calculo de la matriz de PRECIOS y RUTAS
# PRECIOS - contiene la matriz del mejor precio para ir de un nodo a otro
# RUTAS - contiene los nodos intermedios para ir de un nodo a otro
#####
def Precios(TARIFAS):
#####
#Total de Nodos
N = len(TARIFAS[0])

#Inicialización de la tabla de precios
PRECIOS = [ [9999]*N for i in range(N)] #n x n
RUTA = [ [""]*N for i in range(N)]

#Se recorren todos los nodos con dos bucles(origen - destino)
# para ir construyendo la matriz de PRECIOS
for i in range(N-1):
    for j in range(i+1, N):

        # Asumimos que el mínimo es la tarifa de un nodo a otro inicialmente
        MIN = TARIFAS[i][j]
        print("La tarifa desde " + str(i) + " hasta " + str(j) + " con un precio d

        # Asumimos que la mejor ruta de un nodo a otro comienza en el nodo inicial
        RUTA[i][j] = i

        # Recorremos posibles movimientos intermedios
        for k in range(i, j):

            # Si el precio de ir de i a k más la tarifa de k a j es menor que la ta
```

```

    if PRECIOS[i][k] + TARIFAS[k][j] < MIN:
        MIN = min(MIN, PRECIOS[i][k] + TARIFAS[k][j] )
        RUTA[i][j] = k
        print("Ruta indirecta identificada desde " + str(i) + " hasta " + str(j))

    # Actualizamos precio de i a j como el mínimo (puede ser pasando por k)
    PRECIOS[i][j] = MIN

return PRECIOS, RUTA

```

```

In [61]: PRECIOS, RUTA = Precios(TARIFAS)
          #print(PRECIOS[0][6])

          print("PRECIOS")
          for i in range(len(TARIFAS)):
              print(PRECIOS[i])

          print("\nRUTA")
          for i in range(len(TARIFAS)):
              print(RUTA[i])

```

La tarifa desde 0 hasta 1 con un precio de 5
 La tarifa desde 0 hasta 2 con un precio de 4
 La tarifa desde 0 hasta 3 con un precio de 3
 La tarifa desde 0 hasta 4 con un precio de inf
 Ruta indirecta identificada desde 0 hasta 4 pasando por 1 con un precio de 8
 La tarifa desde 0 hasta 5 con un precio de 999
 Ruta indirecta identificada desde 0 hasta 5 pasando por 2 con un precio de 8
 La tarifa desde 0 hasta 6 con un precio de 999
 Ruta indirecta identificada desde 0 hasta 6 pasando por 1 con un precio de 16
 Ruta indirecta identificada desde 0 hasta 6 pasando por 2 con un precio de 14
 Ruta indirecta identificada desde 0 hasta 6 pasando por 3 con un precio de 12
 Ruta indirecta identificada desde 0 hasta 6 pasando por 5 con un precio de 11
 La tarifa desde 1 hasta 2 con un precio de 999
 La tarifa desde 1 hasta 3 con un precio de 2
 La tarifa desde 1 hasta 4 con un precio de 3
 La tarifa desde 1 hasta 5 con un precio de 999
 Ruta indirecta identificada desde 1 hasta 5 pasando por 3 con un precio de 8
 La tarifa desde 1 hasta 6 con un precio de 11
 Ruta indirecta identificada desde 1 hasta 6 pasando por 4 con un precio de 7
 La tarifa desde 2 hasta 3 con un precio de 1
 La tarifa desde 2 hasta 4 con un precio de 999
 Ruta indirecta identificada desde 2 hasta 4 pasando por 3 con un precio de 6
 La tarifa desde 2 hasta 5 con un precio de 4
 La tarifa desde 2 hasta 6 con un precio de 10
 Ruta indirecta identificada desde 2 hasta 6 pasando por 5 con un precio de 7
 La tarifa desde 3 hasta 4 con un precio de 5
 La tarifa desde 3 hasta 5 con un precio de 6
 La tarifa desde 3 hasta 6 con un precio de 9
 La tarifa desde 4 hasta 5 con un precio de 999
 La tarifa desde 4 hasta 6 con un precio de 4
 La tarifa desde 5 hasta 6 con un precio de 3
 PRECIOS

```

[9999, 5, 4, 3, 8, 8, 11]
[9999, 9999, 999, 2, 3, 8, 7]
[9999, 9999, 9999, 1, 6, 4, 7]
[9999, 9999, 9999, 9999, 5, 6, 9]
[9999, 9999, 9999, 9999, 9999, 999, 4]
[9999, 9999, 9999, 9999, 9999, 9999, 3]
[9999, 9999, 9999, 9999, 9999, 9999, 9999]
  
```

RUTA

```

['', 0, 0, 0, 1, 2, 5]
['', '', 1, 1, 1, 3, 4]
['', '', '', 2, 3, 2, 5]
['', '', '', '', 3, 3, 3]
['', '', '', '', '', 4, 4]
['', '', '', '', '', '', 5]
['', '', '', '', '', '', '']
  
```

```

In [62]: #Calculo de la ruta usando la matriz RUTA
def calcular_ruta(RUTA, desde, hasta):
    if desde == RUTA[desde][hasta]:
        #if desde == hasta:
            #print("Ir a :" + str(desde))
            return desde
        else:
            return str(calcular_ruta(RUTA, desde, RUTA[desde][hasta])) + ',' + str(RUTA[desde][hasta])

print("\nLa ruta es:")
calcular_ruta(RUTA, 0,6)
  
```

La ruta es:

Out[62]: '0,2,5'

Problema de Asignacion de tarea

```
In [63]: #Asignacion de tareas - Ramificación y Poda
#####
#   T A R E A
#   A
#   G
#   E
#   N
#   T
#   E

COSTES=[[11,12,18,40],
        [14,15,13,22],
        [11,17,19,23],
        [17,14,20,28]]
```

```
In [64]: #Calculo del valor de una solucion parcial
def valor(S,COSTES):
    VALOR = 0
    for i in range(len(S)):
        VALOR += COSTES[S[i]][i]
    return VALOR

valor((3,2, ),COSTES)
```

Out[64]: 34

```
In [65]: #Coste inferior para soluciones parciales
# (1,3,) Se asigna la tarea 1 al agente 0 y la tarea 3 al agente 1

def CI(S,COSTES):
    # Calculo el coste parcial de la solución dada
    coste = valor(S,COSTES)

    # Calculo coste mínimo de opciones restantes no usadas permitiendo reutiliza
    no_usadas = set(range(len(COSTES)))-set(S)

    for i in range(len(S),len(COSTES)):
        coste += min([COSTES[i][j] for j in no_usadas])

    return coste

def CS(S,COSTES):
    # Calculo el coste parcial de la solución dada
    coste = valor(S,COSTES)

    # Calculo coste máximo de opciones restantes no usadas
    no_usadas = set(range(len(COSTES)))-set(S)

    for i in range(len(S),len(COSTES)):
```

```

        coste += max([COSTES[i][j] for j in no_usadas])

    return coste

CI((0,1),COSTES)

```

Out[65]: 65

```

In [66]: #Genera tantos hijos como posibilidades haya para la siguiente elemento de
#(0,) -> (0,1), (0,2), (0,3)
def crear_hijos(NODO, N):
    HIJOS = []
    for i in range(N):
        if i not in NODO:
            HIJOS.append({'s':NODO +(i,) })
    return HIJOS

```

In [67]: crear_hijos((0,) , 4)

Out[67]: [{'s': (0, 1)}, {'s': (0, 2)}, {'s': (0, 3)}]

```

In [68]: def ramificacion_y_poda(COSTES):
#Construccion iterativa de soluciones(arbol). En cada etapa asignamos un agente(
#Nodos del grafo { s:(1,2),CI:3,CS:5 }
    print(COSTES)
    DIMENSION = len(COSTES)
    MEJOR_SOLUCION=tuple( i for i in range(len(COSTES)) )
    CotaSup = valor(MEJOR_SOLUCION,COSTES)
    print("Cota Superior:", CotaSup)

    NODOS=[]
    NODOS.append({'s':(), 'ci':CI((),COSTES) })

    iteracion = 0

    while( len(NODOS) > 0):
        iteracion +=1

        nodo_prometedor = [ min(NODOS, key=lambda x:x['ci']) ][0]['s']
        print("Nodo prometedor:", nodo_prometedor)

        #Ramificacion
        #Se generan Los hijos
        HIJOS = [ {'s':x['s'], 'ci':CI(x['s'], COSTES) } for x in crear_hijos(nodo_
        print("Explorando hijos:")
        print(HIJOS)

        #Revisamos la cota superior y nos quedamos con la mejor solucion si llegamos
        NODO_FINAL = [x for x in HIJOS if len(x['s']) == DIMENSION ]
        if len(NODO_FINAL) >0:
            print("\n*****Soluciones:", [x for x in HIJOS if len(x['s']) == DIMENS
            if NODO_FINAL[0]['ci'] < CotaSup:
                CotaSup = NODO_FINAL[0]['ci']
                MEJOR_SOLUCION = NODO_FINAL

        #Poda
        HIJOS = [x for x in HIJOS if x['ci'] < CotaSup ]
        print("Despues de la poda:")
        print(HIJOS)

```

```
#Añadimos los hijos
NODOS.extend(HIJOS)

#Eliminamos el nodo ramificado
NODOS = [ x for x in NODOS if x['s'] != nodo_prometedor ]
print("NODOS a explorar: ")
print(NODOS)

print("La solución final es:" ,MEJOR_SOLUCION , " en " , iteracion , " iteraci

ramificacion_y_poda(COSTES)
```

```

[[11, 12, 18, 40], [14, 15, 13, 22], [11, 17, 19, 23], [17, 14, 20, 28]]
Cota Superior: 73
Nodo prometedor: ()
Explorando hijos:
[{'s': (0,), 'ci': 55}, {'s': (1,), 'ci': 55}, {'s': (2,), 'ci': 50}, {'s': (3,), 'ci': 55}]
Despues de la poda:
[{'s': (0,), 'ci': 55}, {'s': (1,), 'ci': 55}, {'s': (2,), 'ci': 50}, {'s': (3,), 'ci': 55}]
NODOS a explorar:
[{'s': (0,), 'ci': 55}, {'s': (1,), 'ci': 55}, {'s': (2,), 'ci': 50}, {'s': (3,), 'ci': 55}]
Nodo prometedor: (2,)
Explorando hijos:
[{'s': (2, 0), 'ci': 54}, {'s': (2, 1), 'ci': 54}, {'s': (2, 3), 'ci': 50}]
Despues de la poda:
[{'s': (2, 0), 'ci': 54}, {'s': (2, 1), 'ci': 54}, {'s': (2, 3), 'ci': 50}]
NODOS a explorar:
[{'s': (0,), 'ci': 55}, {'s': (1,), 'ci': 55}, {'s': (3,), 'ci': 55}, {'s': (2, 0), 'ci': 54}, {'s': (2, 1), 'ci': 54}, {'s': (2, 3), 'ci': 50}]
Nodo prometedor: (2, 3)
Explorando hijos:
[{'s': (2, 3, 0), 'ci': 57}, {'s': (2, 3, 1), 'ci': 55}]
Despues de la poda:
[{'s': (2, 3, 0), 'ci': 57}, {'s': (2, 3, 1), 'ci': 55}]
NODOS a explorar:
[{'s': (0,), 'ci': 55}, {'s': (1,), 'ci': 55}, {'s': (3,), 'ci': 55}, {'s': (2, 0), 'ci': 54}, {'s': (2, 1), 'ci': 54}, {'s': (2, 3, 0), 'ci': 57}, {'s': (2, 3, 1), 'ci': 55}]
Nodo prometedor: (2, 0)
Explorando hijos:
[{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 0, 3), 'ci': 57}]
Despues de la poda:
[{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 0, 3), 'ci': 57}]
NODOS a explorar:
[{'s': (0,), 'ci': 55}, {'s': (1,), 'ci': 55}, {'s': (3,), 'ci': 55}, {'s': (2, 1), 'ci': 54}, {'s': (2, 3, 0), 'ci': 57}, {'s': (2, 3, 1), 'ci': 55}, {'s': (2, 0, 1), 'ci': 64}, {'s': (2, 0, 3), 'ci': 57}]
Nodo prometedor: (2, 1)
Explorando hijos:
[{'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}]
Despues de la poda:
[{'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}]
NODOS a explorar:
[{'s': (0,), 'ci': 55}, {'s': (1,), 'ci': 55}, {'s': (3,), 'ci': 55}, {'s': (2, 3, 0), 'ci': 57}, {'s': (2, 3, 1), 'ci': 55}, {'s': (2, 0, 1), 'ci': 64}, {'s': (2, 0, 3), 'ci': 57}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}]
Nodo prometedor: (0,)
Explorando hijos:
[{'s': (0, 1), 'ci': 65}, {'s': (0, 2), 'ci': 59}, {'s': (0, 3), 'ci': 56}]
Despues de la poda:
[{'s': (0, 1), 'ci': 65}, {'s': (0, 2), 'ci': 59}, {'s': (0, 3), 'ci': 56}]
NODOS a explorar:
[{'s': (1,), 'ci': 55}, {'s': (3,), 'ci': 55}, {'s': (2, 3, 0), 'ci': 57}, {'s': (2, 3, 1), 'ci': 55}, {'s': (2, 0, 1), 'ci': 64}, {'s': (2, 0, 3), 'ci': 57}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (0, 2), 'ci': 59}, {'s': (0, 3), 'ci': 56}]
Nodo prometedor: (1,)
Explorando hijos:
[{'s': (1, 0), 'ci': 65}, {'s': (1, 2), 'ci': 59}, {'s': (1, 3), 'ci': 56}]

```

Despues de la poda:

```
[{'s': (1, 0), 'ci': 65}, {'s': (1, 2), 'ci': 59}, {'s': (1, 3), 'ci': 56}]
```

NODOS a explorar:

```
[{'s': (3, 0), 'ci': 55}, {'s': (2, 3, 0), 'ci': 57}, {'s': (2, 3, 1), 'ci': 55},
{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 0, 3), 'ci': 57}, {'s': (2, 1, 0), 'ci': 72},
{'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (0, 2), 'ci': 59},
{'s': (0, 3), 'ci': 56}, {'s': (1, 0), 'ci': 65}, {'s': (1, 2), 'ci': 59}, {'s': (1, 3), 'ci': 56}]
```

Nodo prometedor: (3, 0)

Explorando hijos:

```
[{'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (3, 2), 'ci': 59}]
```

Despues de la poda:

```
[{'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (3, 2), 'ci': 59}]
```

NODOS a explorar:

```
[{'s': (2, 3, 0), 'ci': 57}, {'s': (2, 3, 1), 'ci': 55}, {'s': (2, 0, 1), 'ci': 64},
{'s': (2, 0, 3), 'ci': 57}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63},
{'s': (0, 1), 'ci': 65}, {'s': (0, 2), 'ci': 59}, {'s': (0, 3), 'ci': 56},
{'s': (1, 0), 'ci': 65}, {'s': (1, 2), 'ci': 59}, {'s': (1, 3), 'ci': 56},
{'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (3, 2), 'ci': 59}]
```

Nodo prometedor: (2, 3, 1)

Explorando hijos:

```
[{'s': (2, 3, 1, 0), 'ci': 78}]
```

*****Soluciones: [{'s': (2, 3, 1, 0), 'ci': 78}]

Despues de la poda:

```
[]
```

NODOS a explorar:

```
[{'s': (2, 3, 0), 'ci': 57}, {'s': (2, 0, 1), 'ci': 64}, {'s': (2, 0, 3), 'ci': 57},
{'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65},
{'s': (0, 2), 'ci': 59}, {'s': (0, 3), 'ci': 56}, {'s': (1, 0), 'ci': 65},
{'s': (1, 2), 'ci': 59}, {'s': (1, 3), 'ci': 56}, {'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (3, 2), 'ci': 59}]
```

Nodo prometedor: (0, 3)

Explorando hijos:

```
[{'s': (0, 3, 1), 'ci': 58}, {'s': (0, 3, 2), 'ci': 58}]
```

Despues de la poda:

```
[{'s': (0, 3, 1), 'ci': 58}, {'s': (0, 3, 2), 'ci': 58}]
```

NODOS a explorar:

```
[{'s': (2, 3, 0), 'ci': 57}, {'s': (2, 0, 1), 'ci': 64}, {'s': (2, 0, 3), 'ci': 57},
{'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65},
{'s': (0, 2), 'ci': 59}, {'s': (1, 0), 'ci': 65}, {'s': (1, 2), 'ci': 59},
{'s': (1, 3), 'ci': 56}, {'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (3, 2), 'ci': 59},
{'s': (0, 3, 1), 'ci': 58}, {'s': (0, 3, 2), 'ci': 58}]
```

Nodo prometedor: (1, 3)

Explorando hijos:

```
[{'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
```

Despues de la poda:

```
[{'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
```

NODOS a explorar:

```
[{'s': (2, 3, 0), 'ci': 57}, {'s': (2, 0, 1), 'ci': 64}, {'s': (2, 0, 3), 'ci': 57},
{'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65},
{'s': (0, 2), 'ci': 59}, {'s': (1, 0), 'ci': 65}, {'s': (1, 2), 'ci': 59},
{'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (3, 2), 'ci': 59},
{'s': (0, 3, 1), 'ci': 58}, {'s': (0, 3, 2), 'ci': 58}, {'s': (1, 3, 0), 'ci': 66},
{'s': (1, 3, 2), 'ci': 64}]
```

Nodo prometedor: (2, 3, 0)

Explorando hijos:

```
[{'s': (2, 3, 0, 1), 'ci': 65}]
```

*****Soluciones: [{'s': (2, 3, 0, 1), 'ci': 65}]

Despues de la poda:

[]

NODOS a explorar:

```
{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 0, 3), 'ci': 57}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (0, 2), 'ci': 59}, {'s': (1, 0), 'ci': 65}, {'s': (1, 2), 'ci': 59}, {'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (3, 2), 'ci': 59}, {'s': (0, 3, 1), 'ci': 58}, {'s': (0, 3, 2), 'ci': 58}, {'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
```

Nodo prometedor: (2, 0, 3)

Explorando hijos:

```
{'s': (2, 0, 3, 1), 'ci': 65}]
```

*****Soluciones: [{'s': (2, 0, 3, 1), 'ci': 65}]

Despues de la poda:

[]

NODOS a explorar:

```
{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (0, 2), 'ci': 59}, {'s': (1, 0), 'ci': 65}, {'s': (1, 2), 'ci': 59}, {'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (3, 2), 'ci': 59}, {'s': (0, 3, 1), 'ci': 58}, {'s': (0, 3, 2), 'ci': 58}, {'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
```

Nodo prometedor: (0, 3, 1)

Explorando hijos:

```
{'s': (0, 3, 1, 2), 'ci': 61}]
```

*****Soluciones: [{'s': (0, 3, 1, 2), 'ci': 61}]

Despues de la poda:

[]

NODOS a explorar:

```
{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (0, 2), 'ci': 59}, {'s': (1, 0), 'ci': 65}, {'s': (1, 2), 'ci': 59}, {'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (3, 2), 'ci': 59}, {'s': (0, 3, 2), 'ci': 58}, {'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
```

Nodo prometedor: (0, 3, 2)

Explorando hijos:

```
{'s': (0, 3, 2, 1), 'ci': 66}]
```

*****Soluciones: [{'s': (0, 3, 2, 1), 'ci': 66}]

Despues de la poda:

[]

NODOS a explorar:

```
{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (0, 2), 'ci': 59}, {'s': (1, 0), 'ci': 65}, {'s': (1, 2), 'ci': 59}, {'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (3, 2), 'ci': 59}, {'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
```

Nodo prometedor: (0, 2)

Explorando hijos:

```
{'s': (0, 2, 1), 'ci': 69}, {'s': (0, 2, 3), 'ci': 62}]
```

Despues de la poda:

[]

NODOS a explorar:

```
{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (1, 0), 'ci': 65}, {'s': (1, 2), 'ci': 59}, {'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (3, 2), 'ci': 59}, {'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
```

Nodo prometedor: (1, 2)

Explorando hijos:

```
{'s': (1, 2, 0), 'ci': 77}, {'s': (1, 2, 3), 'ci': 68}]
```

Despues de la poda:

```

[]
NODOS a explorar:
[{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (1, 0), 'ci': 65}, {'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (3, 2), 'ci': 59}, {'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
Nodo prometedor: (3, 2)
Explorando hijos:
[{'s': (3, 2, 0), 'ci': 66}, {'s': (3, 2, 1), 'ci': 64}]
Despues de la poda:
[]
NODOS a explorar:
[{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (1, 0), 'ci': 65}, {'s': (3, 0), 'ci': 60}, {'s': (3, 1), 'ci': 60}, {'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
Nodo prometedor: (3, 0)
Explorando hijos:
[{'s': (3, 0, 1), 'ci': 62}, {'s': (3, 0, 2), 'ci': 62}]
Despues de la poda:
[]
NODOS a explorar:
[{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (1, 0), 'ci': 65}, {'s': (3, 1), 'ci': 60}, {'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
Nodo prometedor: (3, 1)
Explorando hijos:
[{'s': (3, 1, 0), 'ci': 70}, {'s': (3, 1, 2), 'ci': 68}]
Despues de la poda:
[]
NODOS a explorar:
[{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 1, 0), 'ci': 72}, {'s': (2, 1, 3), 'ci': 63}, {'s': (0, 1), 'ci': 65}, {'s': (1, 0), 'ci': 65}, {'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
Nodo prometedor: (2, 1, 3)
Explorando hijos:
[{'s': (2, 1, 3, 0), 'ci': 86}]

*****Soluciones: [{'s': (2, 1, 3, 0), 'ci': 86}]
Despues de la poda:
[]
NODOS a explorar:
[{'s': (2, 0, 1), 'ci': 64}, {'s': (2, 1, 0), 'ci': 72}, {'s': (0, 1), 'ci': 65}, {'s': (1, 0), 'ci': 65}, {'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
Nodo prometedor: (2, 0, 1)
Explorando hijos:
[{'s': (2, 0, 1, 3), 'ci': 64}]

*****Soluciones: [{'s': (2, 0, 1, 3), 'ci': 64}]
Despues de la poda:
[]
NODOS a explorar:
[{'s': (2, 1, 0), 'ci': 72}, {'s': (0, 1), 'ci': 65}, {'s': (1, 0), 'ci': 65}, {'s': (1, 3, 0), 'ci': 66}, {'s': (1, 3, 2), 'ci': 64}]
Nodo prometedor: (1, 3, 2)
Explorando hijos:
[{'s': (1, 3, 2, 0), 'ci': 87}]

*****Soluciones: [{'s': (1, 3, 2, 0), 'ci': 87}]
Despues de la poda:
[]

```

```

NODOS a explorar:
[{'s': (2, 1, 0), 'ci': 72}, {'s': (0, 1), 'ci': 65}, {'s': (1, 0), 'ci': 65},
{'s': (1, 3, 0), 'ci': 66}]
Nodo prometedor: (0, 1)
Explorando hijos:
[{'s': (0, 1, 2), 'ci': 73}, {'s': (0, 1, 3), 'ci': 66}]
Despues de la poda:
[]
NODOS a explorar:
[{'s': (2, 1, 0), 'ci': 72}, {'s': (1, 0), 'ci': 65}, {'s': (1, 3, 0), 'ci': 66}]
Nodo prometedor: (1, 0)
Explorando hijos:
[{'s': (1, 0, 2), 'ci': 73}, {'s': (1, 0, 3), 'ci': 66}]
Despues de la poda:
[]
NODOS a explorar:
[{'s': (2, 1, 0), 'ci': 72}, {'s': (1, 3, 0), 'ci': 66}]
Nodo prometedor: (1, 3, 0)
Explorando hijos:
[{'s': (1, 3, 0, 2), 'ci': 69}]

*****Soluciones: [{'s': (1, 3, 0, 2), 'ci': 69}]
Despues de la poda:
[]
NODOS a explorar:
[{'s': (2, 1, 0), 'ci': 72}]
Nodo prometedor: (2, 1, 0)
Explorando hijos:
[{'s': (2, 1, 0, 3), 'ci': 72}]

*****Soluciones: [{'s': (2, 1, 0, 3), 'ci': 72}]
Despues de la poda:
[]
NODOS a explorar:
[]
La solucion final es: [{'s': (0, 3, 1, 2), 'ci': 61}] en 27 iteraciones para
dimension: 4

```

Descenso del gradiente

```

In [69]: import math                                #Funciones matematicas
import matplotlib.pyplot as plt                    #Generacion de gráficos (otra opcion seaborn)
import numpy as np                                #Tratamiento matriz N-dimensionales y otras (fu
import scipy as sc
import random

```

Vamos a buscar el minimo de la funcion paraboloide :

$$f(x) = x^2 + y^2$$

Obviamente se encuentra en (x,y)=(0,0) pero probaremos como llegamos a él a través del descenso del gradiente.

```

In [70]: #Definimos la funcion
#Paraboloide
f = lambda X:      X[0]**2 + X[1]**2      #Funcion

```

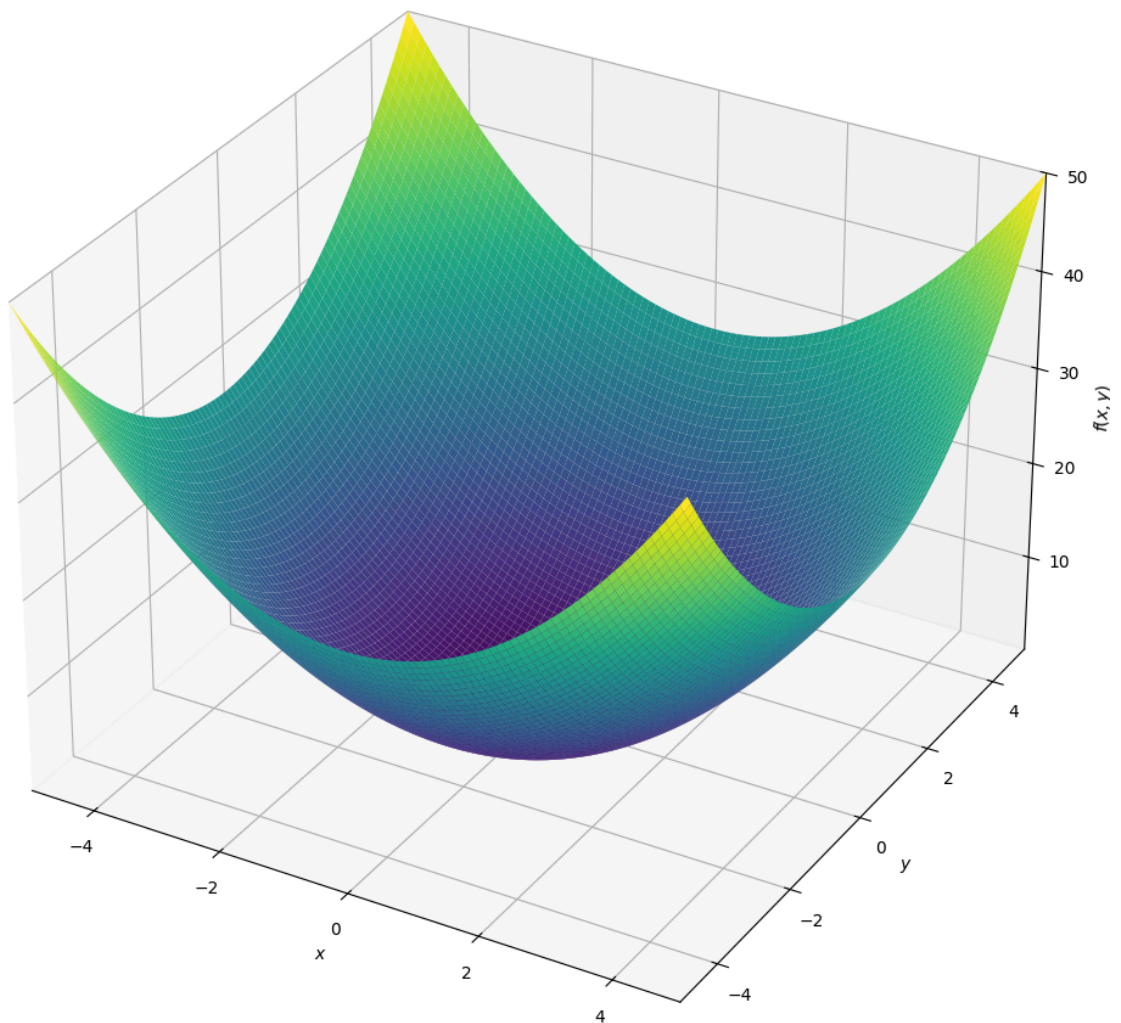
```
df = lambda X: [2*X[0] , 2*X[1]]           #Gradiente

df([1,2])
```

Out[70]: [2, 4]

```
In [71]: from sympy import symbols
from sympy.plotting import plot
from sympy.plotting import plot3d
x,y = symbols('x y')
plot3d(x**2 + y**2,
      (x,-5,5),(y,-5,5),
      title='x**2 + y**2',
      size=(10,10))
```

$x^2 + y^2$



Out[71]: <sympy.plotting.backends.matplotlibbackend.matplotlib.MatplotlibBackend at 0x1e048bbc190>

```
In [72]: #Prepara Los datos para dibujar mapa de niveles de Z
resolucion = 100
rango=5.5

X=np.linspace(-rango,rango,resolucion)
Y=np.linspace(-rango,rango,resolucion)
```

```

Z=np.zeros((resolucion,resolucion))
for ix,x in enumerate(X):
    for iy,y in enumerate(Y):
        Z[iy,ix] = f([x,y])

#Pinta el mapa de niveles de Z
plt.contourf(X,Y,Z,resolucion)
plt.colorbar()

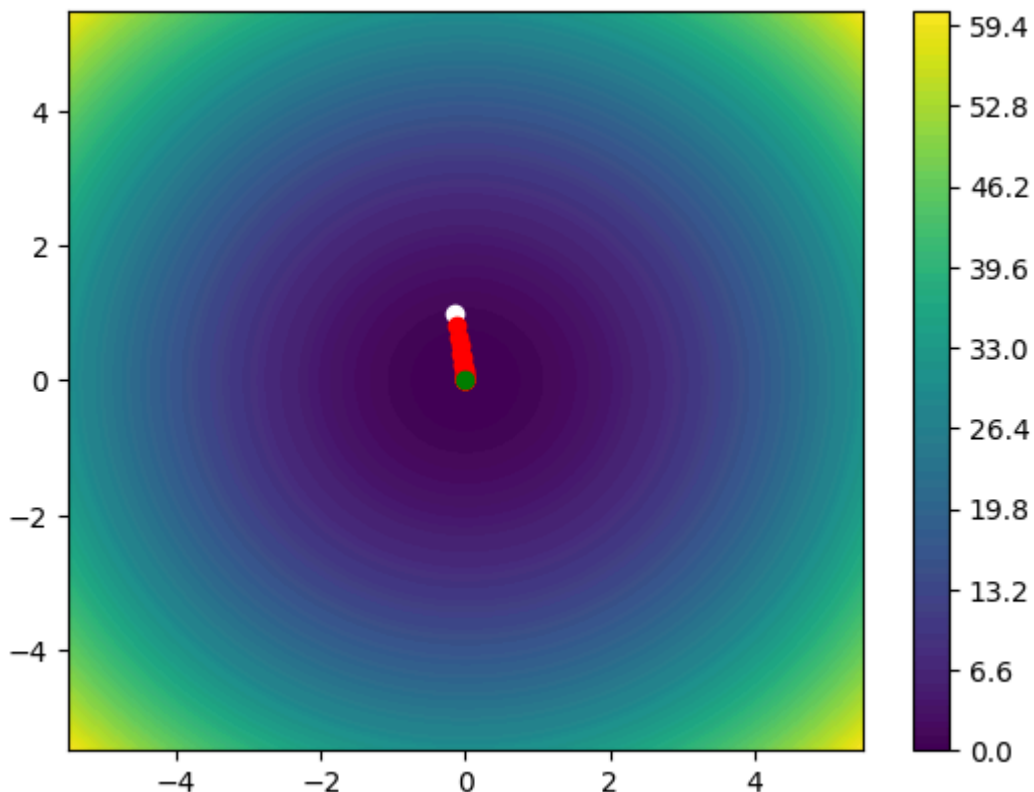
#Generamos un punto aleatorio inicial y pintamos de blanco
P=[random.uniform(-5,5 ),random.uniform(-5,5 ) ]
plt.plot(P[0],P[1],"o",c="white")

#Tasa de aprendizaje. Fija. Sería más efectivo reducirlo a medida que nos acerca
TA=.1

#Iteraciones:50
for _ in range(50):
    grad = df(P)
    #print(P,grad)
    P[0],P[1] = P[0] - TA*grad[0] , P[1] - TA*grad[1]
    plt.plot(P[0],P[1],"o",c="red")

#Dibujamos el punto final y pintamos de verde
plt.plot(P[0],P[1],"o",c="green")
plt.show()
print("Solucion:" , P , f(P))

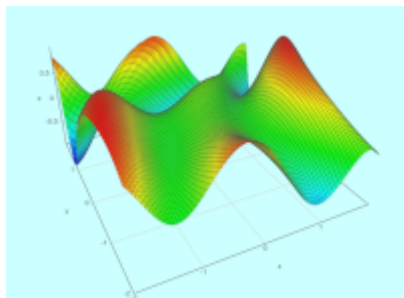
```



Solucion: [-2.2118974845577934e-06, 1.4342378545029996e-05] 2.1059631281112984e-10

¿Te atreves a optimizar la función?:

$$f(x) = \sin(1/2 * x^2 - 1/4 * y^2 + 3) * \cos(2 * x + 1 - e^y)$$



In [73]: *#Definimos la funcion*

```
f= lambda X: math.sin(1/2 * X[0]**2 - 1/4 * X[1]**2 + 3) *math.cos(2*X[0] + 1 -
```

In [74]: resultado = f([1, 0])

```
print(resultado)
```

0.14597733051683126

In [75]: from sympy import symbols, sin, cos, exp, Rational

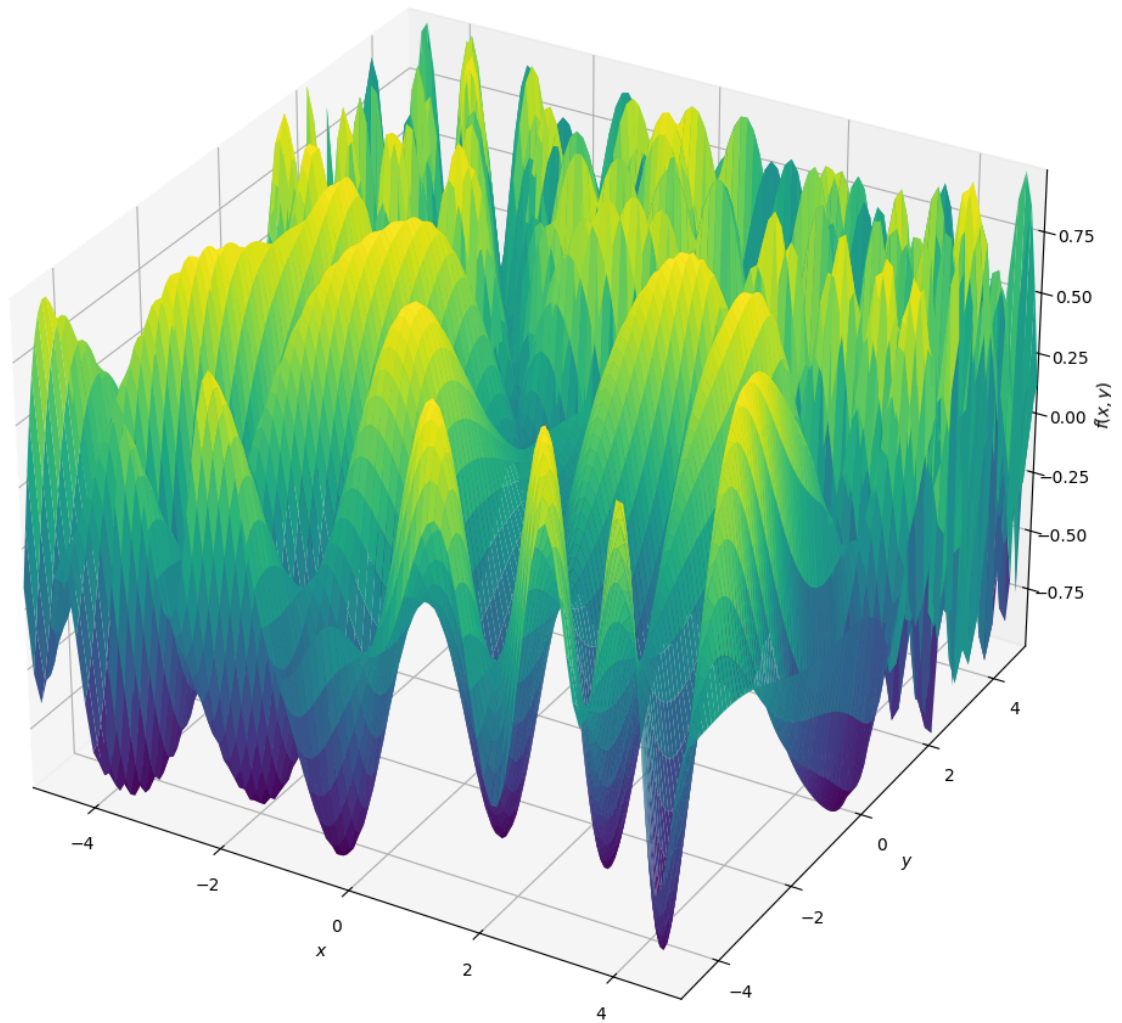
```
from sympy.plotting import plot3d
```

```
x, y = symbols('x y')
```

```
expr = sin(Rational(1,2)*x**2 - Rational(1,4)*y**2 + 3) * cos(2*x + 1 - exp(y))
```

```
plot3d(expr,
        (x, -5, 5), (y, -5, 5),
        title='f(x,y) = sin(0.5x^2 - 0.25y^2 + 3) * cos(2x+1-e^y)',
        size=(10, 10))
```

$$f(x,y) = \sin(0.5x^2 - 0.25y^2 + 3) * \cos(2x+1-e^y)$$



Out[75]: <sympy.plotting.backends.matplotlibbackend.matplotlib.MatplotlibBackend at 0x1e048b5d6d0>

```
In [78]: import numpy as np
import matplotlib.pyplot as plt
import random
import math

# ---- Función original ----
f = lambda X: math.sin(0.5*X[0]**2 - 0.25*X[1]**2 + 3) * math.cos(2*X[0] + 1 - math.exp(X[1]))

# ---- Gradiente analítico ----
# f(x,y) = sin(g(x,y)) * cos(h(x,y))
# g = 0.5x^2 - 0.25y^2 + 3      -> dg/dx = x      , dg/dy = -0.5y
# h = 2x + 1 - e^y            -> dh/dx = 2      , dh/dy = -e^y
#
# df/dx = cos(g)*dg/dx*cos(h) - sin(g)*sin(h)*dh/dx
# df/dy = cos(g)*dg/dy*cos(h) - sin(g)*sin(h)*dh/dy
def df(X):
    x, y = X[0], X[1]
    g = 0.5*x**2 - 0.25*y**2 + 3
    h = 2*x + 1 - math.exp(y)

    dgdx, dgdy = x, -0.5*y
```

```

dhdxdx, dhdy = 2, -math.exp(y)

dfdx = math.cos(g)*dgdx*math.cos(h) - math.sin(g)*math.sin(h)*dhdxdx
dfdy = math.cos(g)*dgdy*math.cos(h) - math.sin(g)*math.sin(h)*dhdy

return [dfdx, dfdy]

# ---- Prepara Los datos para dibujar mapa de niveles de Z ----
resolucion = 100
rango = 5.5
X = np.linspace(-rango, rango, resolucion)
Y = np.linspace(-rango, rango, resolucion)
Z = np.zeros((resolucion, resolucion))

for ix, x in enumerate(X):
    for iy, y in enumerate(Y):
        Z[iy, ix] = f([x, y])

# ---- Pinta el mapa de niveles de Z ----
plt.figure(figsize=(8, 8))
plt.contourf(X, Y, Z, resolucion, cmap='viridis')
plt.colorbar()

# ---- Punto aleatorio inicial (blanco) ----
P = [random.uniform(-rango, rango), random.uniform(-rango, rango)]
plt.plot(P[0], P[1], "o", c="white")

# ---- Tasa de aprendizaje (fija) ----
TA = 0.1

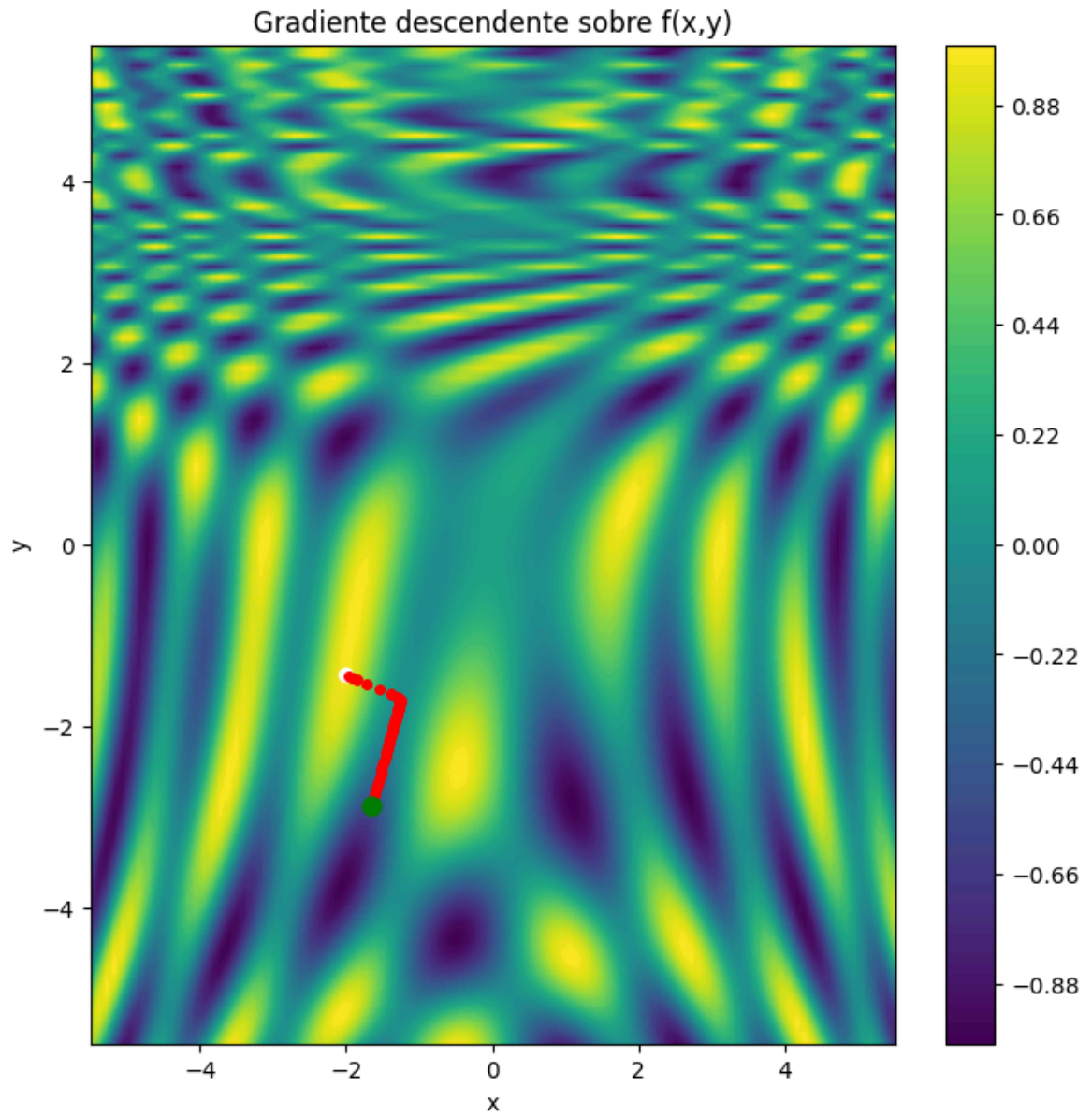
# ---- Iteraciones: gradiente descendente ----
for _ in range(50):
    grad = df(P)
    P[0], P[1] = P[0] - TA*grad[0], P[1] - TA*grad[1]
    plt.plot(P[0], P[1], "o", c="red", markersize=4)

# ---- Punto final (verde) ----
plt.plot(P[0], P[1], "o", c="green", markersize=8)

plt.title("Gradiente descendente sobre f(x,y)")
plt.xlabel("x")
plt.ylabel("y")
plt.show()

print("Solucion:", P, f(P))

```



Solucion: $[-1.6584998841911343, -2.881738320291937] -0.5364786422585749$